

내장함수

■ 내장함수

- 파이썬 내부에 미리 정의되어 있는 함수
- 외부모듈과는 달리 import 과정없이 사용 가능
ex) print(), type(), len(), list(), map(), set(), ...
- `__builtins__` 속성을 통해서 내장함수 목록 확인 가능

```
dir(__builtins__)
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '__build_class__', '__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

■ 내장함수

● 수학 / 계산 관련

함수명	설명
abs(x)	숫자의 절댓값 반환
divmod(a, b)	(몫, 나머지) 튜플로 반환
max(iterable)	최댓값 반환
min(iterable)	최솟값 반환
pow(x, y)	x의 y제곱 반환
round(x, n)	반올림 (소수점 n자리까지)
sum(iterable)	합계 반환

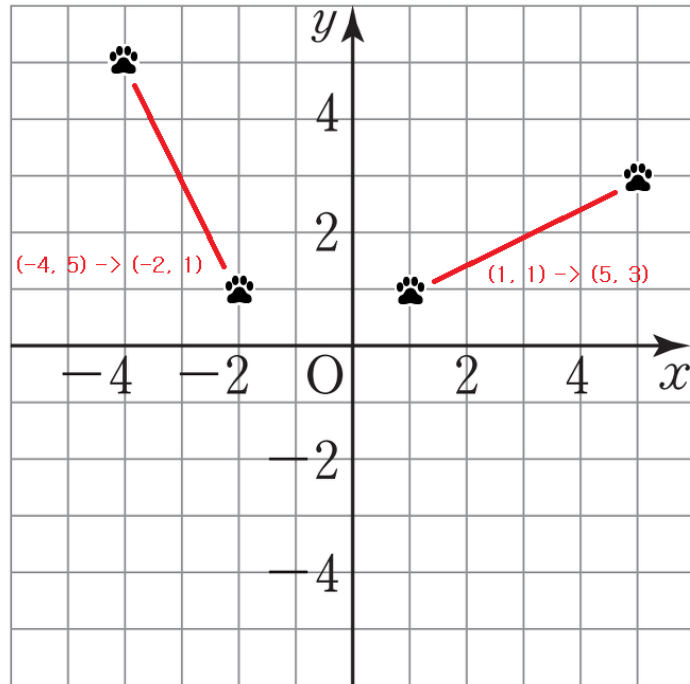
■ 내장함수

● 수학 / 계산 관련

```
print(abs(-7)) # 7
print(divmod(7, 3)) # (2, 1)
scores = [90, 80, 70]
print(max(scores)) # 90
print(min(scores)) # 70
print(sum(scores)) # 240
```

■ 연습문제

- 그래프와 같이 이동한 거리 dx, dy 를 구할 수 있는 distance() 함수 작성



$(-4, 5) \rightarrow (-2, 1)$
 $dx \rightarrow 2, dy \rightarrow 4$

$(1, 1) \rightarrow (5, 3)$
 $dx \rightarrow 4, dy \rightarrow 2$

```
def distance(start, end):  
    # 코드 작성  
  
start = (1, 1)  
end = (5, 3)  
dx, dy = distance(start, end)  
print('dx:', dx, 'dy:', dy)
```

dx: 4 dy: 2

■ 연습문제

- a를 b로 나눈 몫과 나머지 구하기

```
a = 123
```

```
b = 16
```

```
# 코드 작성
```

```
print('몫', quotient)
```

```
print('나머지', remain)
```

```
몫 7  
나머지 11
```

■ 내장함수

● 형 변환 관련

함수명	설명
int(x)	정수로 변환
float(x)	실수로 변환
str(x)	문자열로 변환
list(x)	리스트로 변환
tuple(x)	튜플로 변환
set(x)	집합으로 변환
dict(x)	딕셔너리로 변환

■ 내장함수

● 형 변환 관련

```
print(int("10")) # 10
print(float("3.14")) # 3.14
print(str(123)) # "123"
print(list("abc")) # ['a', 'b', 'c']
print(tuple([1,2,3])) # (1,2,3)
```


■ 연습문제

● 문자열 "3.14"를 정수와 실수로 변환하기

```
s = "3.14"
```

```
# 코드 작성
```

```
실수: 3.14
```

```
정수: 3
```

● 문자열 "Python"을 리스트로 변환하기

```
s = "Python"
```

```
# 코드 작성
```

```
['P', 'y', 't', 'h', 'o', 'n']
```

■ 내장함수

● 순서 / 반복 관련

함수명	설명
<code>range(start, stop, step)</code>	범위 객체 생성 (반복문에서 사용)
<code>enumerate(iterable)</code>	(인덱스, 값) 쌍을 반환
<code>zip(*iterables)</code>	여러 시퀀스를 묶어 튜플로 반환
<code>sorted(iterable)</code>	정렬된 리스트 반환
<code>reversed(iterable)</code>	역순 반복자 반환

■ 내장함수

● 순서 / 반복 관련

```
for i in range(1, 5):  
    print(i) # 1, 2, 3, 4  
  
fruits = ["apple", "banana"]  
for idx, fruit in enumerate(fruits, start=1):  
    print(idx, fruit)  
  
names = ["철수", "영희"]  
scores = [90, 85]  
for name, score in zip(names, scores):  
    print(name, score)  
  
print(sorted([3,1,2])) # [1,2,3]
```

■ 연습문제

- 리스트 ['사과', '바나나', '포도']를 번호와 함께 출력하기

```
fruits = ['사과', '바나나', '포도']
```

```
# 코드 작성
```

- 1 사과
- 2 바나나
- 3 포도

■ 연습문제

- 주어진 데이터를 **결과**와 같은 형식의 데이터로 저장하기

데이터

```
fish1_length = [  
    25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7,  
    31.0, 31.0, 31.5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5,  
    34.0, 34.0, 34.5, 35.0, 35.0, 35.0, 35.0, 36.0, 36.0, 37.0,  
    38.5, 38.5, 39.5, 41.0, 41.0  
]  
fish1_weight = [  
    242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0,  
    500.0, 475.0, 500.0, 500.0, 340.0, 600.0, 600.0, 700.0, 700.0,  
    610.0, 650.0, 575.0, 685.0, 620.0, 680.0, 700.0, 725.0, 720.0,  
    714.0, 850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0  
]  
fish2_length = [  
    9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2,  
    12.4, 13.0, 14.3, 15.0  
]  
fish2_weight = [  
    6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2,  
    13.4, 12.2, 19.7, 19.9  
]
```

결과

길이	무게
[25.4, 242.0],	
[26.3, 290.0],	
.	.
.	.
.	.
[15.0, 19.9]	

■ 내장함수

● 논리 / 검사 관련

함수명	설명
all(iterable)	모두 True이면 True
any(iterable)	하나라도 True이면 True
isinstance(obj, class)	자료형 확인
issubclass(cls, classinfo)	상속 여부 확인
type(obj)	객체의 자료형 반환
id(obj)	객체의 메모리 주소 반환

■ 내장함수

● 논리 / 검사 관련

```
print(all([True, 1, "hi"])) # True
print(any([0, "", None])) # False

x = 10
print(isinstance(x, int)) # True
print(type(x)) # <class 'int'>
```

■ 내장함수

● 문자 / 코드 관련

함수명	설명
chr(i)	정수를 해당 유니코드 문자로 변환
ord(c)	문자의 유니코드 정수 반환
hex(x)	16진수 문자열 반환
oct(x)	8진수 문자열 반환

■ 내장함수

● 문자 / 코드 관련

```
print(chr(65)) # 'A'  
print(ord('A')) # 65  
print(hex(255)) # '0xff'  
print(oct(8)) # '0o10'
```

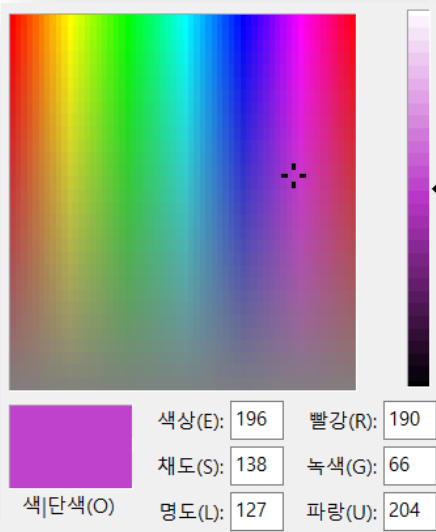
■ 연습문제

- 색상표를 통해 확인된 숫자를 16진수로 표현

```
red = 190  
green = 66  
blue = 205
```

코드 작성

```
print(r, g, b)
```



0xbe 0x42 0xcd

■ 연습문제

- 16진수 문자 코드값을 문자로 변환하기

```
hexa_string = '48 45 4C 4C 4F'  
# 코드 작성
```

HELLO

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char
32	20	[SPACE]	64	40	@
33	21	!	65	41	A
34	22	"	66	42	B
35	23	#	67	43	C
36	24	\$	68	44	D
37	25	%	69	45	E
38	26	&	70	46	F
39	27	'	71	47	G
40	28	(	72	48	H
41	29	)	73	49	I
42	2A	*	74	4A	J
43	2B	+	75	4B	K
44	2C	,	76	4C	L
45	2D	-	77	4D	M
46	2E	.	78	4E	N
47	2F	/	79	4F	O
48	30	0	80	50	P
49	31	1	81	51	Q
50	32	2	82	52	R
51	33	3	83	53	S
52	34	4	84	54	T
53	35	5	85	55	U
54	36	6	86	56	V
55	37	7	87	57	W
56	38	8	88	58	X
57	39	9	89	59	Y
58	3A	:	90	5A	Z
59	3B	;	91	5B	[
60	3C	<	92	5C	\
61	3D	=	93	5D	]
62	3E	>	94	5E	^
63	3F	?	95	5F	_

■ 내장함수

● 입출력 / 시스템 관련

함수명	설명
print(x)	화면에 출력
input()	사용자 입력 받기
open(filename, mode)	파일 열기
dir(obj)	객체가 가진 속성과 메소드 목록 반환

■ 내장함수

● 입출력 / 시스템 관련

```
name = input("이름 입력: ")
print("안녕하세요,", name)

f = open("test.txt", "w")
f.write("Hello Python")
f.close()

print(dir([])) # 리스트가 가진 메소드 목록
```

■ 내장함수

● 기타 / 응용

함수명	설명
eval(str)	문자열을 파이썬 코드처럼 실행
map(func, iterable)	각 요소에 함수를 적용
filter(func, iterable)	조건을 만족하는 요소만 걸러냄

■ 내장함수

● 기타 / 응용

```
print(eval("3+5")) # 8
```

```
nums = [1,2,3,4,5]
```

```
print(list(map(lambda x: x**2, nums)))
```

```
# [1, 4, 9, 16, 25]
```

```
print(list(filter(lambda x: x%2==0, nums)))
```

```
# [2, 4]
```

■ 연습문제

- 리스트 ["1", "20", "3", "40"]를 정수로 변환한 뒤, 10 이상인 숫자만 출력

```
nums = ["1", "20", "3", "40"]
```

```
# 코드 작성
```

```
[20, 40]
```

- 리스트 [5, 10, 15, 20, 25]에서 3의 배수만 제공해서 출력

```
nums = [5, 10, 15, 20, 25]
```

```
# 코드 작성
```

```
[225]
```